

Guía Práctica: Sistema CRUD con JavaFX y PostgreSQL

Materia: Programación Orientada a Objetos

Herramientas: IntelliJ IDEA, Scene Builder, pgAdmin 4, Maven.

1. Configuración de la Base de Datos (PostgreSQL)

Para iniciar el proyecto, primero debemos estructurar la persistencia de datos. Creamos una base de datos local en pgAdmin 4 llamada `practica_00` y ejecutamos el siguiente script para generar la tabla.

Script SQL:

SQL

```
CREATE TABLE estudiantes (  
    id SERIAL PRIMARY KEY,  
    cedula VARCHAR(10) UNIQUE NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    carrera VARCHAR(50) NOT NULL,  
    activo BOOLEAN DEFAULT TRUE  
);
```

```
create table estudiantes(  
    id serial primary key,  
    cedula varchar(10) unique not null,  
    nombre varchar(100) not null,  
    carrera varchar(50) not null,  
    activo boolean default true  
);  
  
insert into estudiantes (cedula, nombre, carrera, activo) values  
    ('1723456789', 'Carlos Andrade', 'Software', true),  
    ('1798765432', 'Ana Flores', 'Redes', false);  
  
select * from estudiantes;  
|
```

2. Dependencia de Maven (`pom.xml`)

Para que Java pueda comunicarse con PostgreSQL, es obligatorio inyectar el conector (Driver JDBC). Esto evita el error *"No suitable driver found"*.

Ubicación: Archivo `pom.xml` (dentro de la etiqueta `<dependencies>`)

Código a pegar:

XML

```
<!-- Source:
https://mvnrepository.com/artifact/org.postgresql/postgresql -->
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.11</version>
  <scope>compile</scope>
</dependency>
```

3. Clase de Conexión (Conexion.java)

Esta clase maneja el puente entre nuestra aplicación y pgAdmin. Aplicamos el pilar del **Encapsulamiento** de POO utilizando `private static` en las credenciales para proteger la información sensible y evitar que sea modificada desde otras partes del sistema.

Ubicación:

src/main/java/com/example/poo_practica_prueba_00/Conexion.java

Código a pegar:

Java

```
package com.example.poo_practica_prueba_00;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Conexion {
    private static final String url =
"jdbc:postgresql://localhost:5432/practica_00";
    private static final String user = "postgres";
    private static final String password = "Iblame2007*";

    public static Connection conectar(){
        try {
            return DriverManager.getConnection(url, user, password);
        } catch (SQLException e){
            System.out.println("Error al conectar: " +
e.getMessage());
            return null;
        }
    }
}
```

4. El Modelo (Estudiante.java)

Creamos la clase entidad que representa a un estudiante en la memoria del programa. Se utilizan constructores, Getters y Setters obligatorios para que el componente `TableView` de JavaFX pueda leer e inyectar los datos en la interfaz gráfica.

Ubicación:

src/main/java/com/example/poo_practica_prueba_00/Estudiante.java **Código a pegar:**

Java

```
package com.example.poo_practica_prueba_00;

public class Estudiante {
    private int id;
    private String cedula;
    private String nombre;
    private String carrera;
    private boolean activo;

    public Estudiante(int id, String cedula, String nombre, String
carrera, boolean activo) {
        this.id = id;
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.activo = activo;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getCedula() {
        return cedula;
    }

    public void setCedula(String cedula) {
        this.cedula = cedula;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getCarrera() {
        return carrera;
    }

    public void setCarrera(String carrera) {
        this.carrera = carrera;
    }
}
```

```
public boolean isActive() {  
    return activo;  
}  
public void setActive(boolean activo) {  
    this.activo = activo;  
}  
}
```

5. Módulo de Login

Consiste en una validación de seguridad inicial utilizando "datos quemados" (hardcoded).

5.1 Vista FXML (`LoginVista.fxml`)

Diseñada en Scene Builder utilizando `TextField` para el usuario y `PasswordField` para ocultar la contraseña.

A screenshot of a login form UI design. The form is centered on a light gray background. At the top, the word "LOGIN" is displayed in a large, bold, black serif font. Below it, the label "Usuario:" is followed by a white text input field with a thin gray border. Underneath that, the label "Contraseña:" is followed by a white password input field with a thin gray border. At the bottom of the form, there is a rectangular button with a light gray background and a thin gray border, containing the word "Ingresar" in a black serif font.

5.2 Controlador (`LoginController.java`)

Evalúa que los campos no estén vacíos y compara las credenciales ingresadas con admin y 1234. Si es exitoso, cierra la ventana actual y levanta la vista del CRUD.

Ubicación:

`src/main/java/com/example/poo_practica_prueba_00/LoginController.java`

Código a pegar:

Java

```

package com.example.poo_practica_prueba_00;

import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import java.io.IOException;

public class LoginController {
    @FXML
    private TextField txtUser;
    @FXML
    private PasswordField txtPassword;

    @FXML
    void onIngresar() {
        String user = txtUser.getText().trim();
        String pass = txtPassword.getText().trim();
        if (user.isEmpty() || pass.isEmpty()) {
            mostrarAlerta("Campos Vacios", "Por favor llena todos los campos.", Alert.AlertType.WARNING);
            return;
        }

        if (user.equals("admin") && pass.equals("1234")) {
            mostrarAlerta("Exito", "Bienvenido al sistema!", Alert.AlertType.INFORMATION);
            abrirCRUD();
        } else {
            mostrarAlerta("Error", "Usuario o contraseña incorrectos", Alert.AlertType.ERROR);
        }
    }

    @FXML
    void mostrarAlerta(String titulo, String mensaje, Alert.AlertType tipo) {
        Alert alerta = new Alert(tipo);
        alerta.setTitle(titulo);
        alerta.setHeaderText(null);
        alerta.setContentText(mensaje);
        alerta.showAndWait();
    }

    @FXML
    void abrirCRUD() {
        try {
            FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource("CrudVista.fxml"));
            Scene scene = new Scene(fxmlLoader.load());
            Stage stage = new Stage();
            stage.setTitle("Gestion de Estudiantes - CRUD");
            stage.setScene(scene);
            stage.show();

            Stage stageLogin = (Stage) txtUser.getScene().getWindow();

```

```

        stageLogin.close();
    } catch (IOException e) {
        System.out.println("Error al abrir el registro: " +
e.getMessage());
    }
}
}

```

6. Módulo CRUD (Gestión de Estudiantes)

El núcleo de la aplicación. Maneja operaciones de Crear, Leer, Actualizar y Eliminar registros conectándose mediante JDBC.

6.1 Vista FXML (CrudVista.fxml)

Contiene los controles básicos (TextField, ComboBox, CheckBox, Button) y una tabla (TableView) para listar los datos. A cada elemento se le asignó un identificador `fx:id` y eventos `On Action` para interactuar con Java.

ID	CEDULA	NOMBRE	CARRERA	ACTIVO
Tabla sin contenido				

6.2 Controlador (CrudController.java)

Maneja toda la lógica del negocio. Aspectos técnicos importantes implementados:

- **Validaciones:** Controla campos nulos (ej: `carrera == null`) y obliga a que la cédula cumpla con exactamente 10 caracteres numéricos (usando expresiones regulares `\\d+`).
- **Gestión de Recursos:** Se utiliza la estructura `try-with-resources` (ej: `try (Connection con = ...)`) para asegurar que la conexión a la base de datos se cierre automáticamente, evitando fugas de memoria.

- **Eventos de Tabla:** Captura el clic del usuario en una fila para cargar automáticamente los datos en el formulario y permitir su edición o eliminación.

Ubicación:

src/main/java/com/example/poo_practica_prueba_00/CrudController.java

Código a pegar:

Java

```
package com.example.poo_practica_prueba_00;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.Statement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class CrudController {
    @FXML
    private TextField txtCedula, txtNombre;
    @FXML
    private ComboBox<String> cmbCarrera;
    @FXML
    private CheckBox chkActivo;
    @FXML
    private TableView<Estudiante> tablaEstudiantes;
    @FXML
    private TableColumn<Estudiante,String> colNombre, colCedula,
colCarrera;
    @FXML
    private TableColumn<Estudiante,Integer> colId;
    @FXML
    private TableColumn<Estudiante, Boolean> colActivo;

    private ObservableList<Estudiante> listaEstudiante;
    private Estudiante estudianteSeleccionado;

    @FXML
    public void initialize() {

        cmbCarrera.setItems(FXCollections.observableArrayList("Desarrollo de
Software", "Redes", "Electromecánica"));

        colId.setCellValueFactory(new PropertyValueFactory<>("id"));
        colNombre.setCellValueFactory(new
PropertyValueFactory<>("nombre"));
        colCedula.setCellValueFactory(new
PropertyValueFactory<>("cedula"));
        colCarrera.setCellValueFactory(new
PropertyValueFactory<>("carrera"));
        colActivo.setCellValueFactory(new
PropertyValueFactory<>("activo"));

        cargarDatosTabla();
    }
}
```

```

    }

    @FXML
    private void cargarDatosTabla() {
        listaEstudiante = FXCollections.observableArrayList();
        String query = "SELECT * FROM estudiantes";

        try (Connection con = Conexion.conectar();
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery(query)) {

            while (rs.next()) {
                listaEstudiante.add(new Estudiante(
                    rs.getInt("id"),
                    rs.getString("cedula"),
                    rs.getString("nombre"),
                    rs.getString("carrera"),
                    rs.getBoolean("activo")
                ));
            }
            tablaEstudiantes.setItems(listaEstudiante);

        } catch (SQLException e) {
            mostrarAlerta("Error", "No se pudieron cargar los datos: "
+ e.getMessage(), Alert.AlertType.ERROR);
        }
    }

    @FXML
    void alSeleccionarFila() {
        estudianteSeleccionado =
tablaEstudiantes.getSelectionModel().getSelectedItem();

        if (estudianteSeleccionado != null) {
            txtCedula.setText(estudianteSeleccionado.getCedula());
            txtNombre.setText(estudianteSeleccionado.getNombre());
            cmbCarrera.setValue(estudianteSeleccionado.getCarrera());
            chkActivo.setSelected(estudianteSeleccionado.isActivo());
        }
    }

    @FXML
    public void onGuardar() {
        String cedula = txtCedula.getText().trim();
        String nombre = txtNombre.getText().trim();
        String carrera = cmbCarrera.getValue();
        boolean activo = chkActivo.isSelected();

        if (cedula.isEmpty() || nombre.isEmpty() || carrera == null) {
            mostrarAlerta("Validacion", "Llene todos los campos.",
Alert.AlertType.WARNING);
            return;
        }
        if (cedula.length() != 10 || !cedula.matches("\\d+")) {
            mostrarAlerta("Validacion", "La cedula no es valida.",
Alert.AlertType.ERROR);
            return;
        }

        String query = "insert into estudiantes (cedula, nombre,
carrera, activo) values (?, ?, ?, ?)";
    }

```



```

try (Connection con = Conexion.conectar();
    PreparedStatement state = con.prepareStatement(query)) {

    state.setString(1, cedula);
    state.setString(2, nombre);
    state.setString(3, carrera);
    state.setBoolean(4, activo);

    state.executeUpdate();
    mostrarAlerta("Éxito", "Estudiante guardado en la base de
datos.", Alert.AlertType.INFORMATION);

    cargarDatosTabla();
    onLimpiar();

} catch (SQLException e) {
    mostrarAlerta("Error BD", e.getMessage(),
Alert.AlertType.ERROR);
}
}

@FXML
void onModificar() {
    if (estudianteSeleccionado == null) {
        mostrarAlerta("Atención", "Primero debes seleccionar un
estudiante de la tabla.", Alert.AlertType.WARNING);
        return;
    }

    String cedula = txtCedula.getText().trim();
    String nombre = txtNombre.getText().trim();
    String carrera = cmbCarrera.getValue();
    boolean activo = chkActivo.isSelected();

    if (cedula.isEmpty() || nombre.isEmpty() || carrera == null ||
cedula.length() != 10) {
        mostrarAlerta("Validación", "Verifica que todos los campos
estén llenos y la cédula tenga 10 dígitos.", Alert.AlertType.WARNING);
        return;
    }

    String query = "UPDATE estudiantes SET cedula = ?, nombre = ?,
carrera = ?, activo = ? WHERE id = ?";

    try (Connection con = Conexion.conectar();
        PreparedStatement pstmt = con.prepareStatement(query)) {

        pstmt.setString(1, cedula);
        pstmt.setString(2, nombre);
        pstmt.setString(3, carrera);
        pstmt.setBoolean(4, activo);
        pstmt.setInt(5, estudianteSeleccionado.getId());

        pstmt.executeUpdate();
        mostrarAlerta("Éxito", "Estudiante modificado
correctamente.", Alert.AlertType.INFORMATION);

        cargarDatosTabla();
        onLimpiar();
        estudianteSeleccionado = null;
    }
}

```

```

        } catch (SQLException e) {
            mostrarAlerta("Error BD", "Error al modificar: " +
e.getMessage(), Alert.AlertType.ERROR);
        }
    }

    @FXML
    void onEliminar() {
        if (estudianteSeleccionado == null) {
            mostrarAlerta("Atención", "Primero debes seleccionar un
estudiante de la tabla.", Alert.AlertType.WARNING);
            return;
        }

        String query = "DELETE FROM estudiantes WHERE id = ?";

        try (Connection con = Conexion.conectar();
            PreparedStatement pstmt = con.prepareStatement(query)) {

            pstmt.setInt(1, estudianteSeleccionado.getId());
            pstmt.executeUpdate();

            mostrarAlerta("Éxito", "Estudiante eliminado.",
Alert.AlertType.INFORMATION);

            cargarDatosTabla();
            onLimpiar();
            estudianteSeleccionado = null;

        } catch (SQLException e) {
            mostrarAlerta("Error BD", "Error al eliminar: " +
e.getMessage(), Alert.AlertType.ERROR);
        }
    }

    @FXML
    public void onLimpiar() {
        txtCedula.clear();
        txtNombre.clear();
        cmbCarrera.setValue(null);
        chkActivo.setSelected(false);
    }

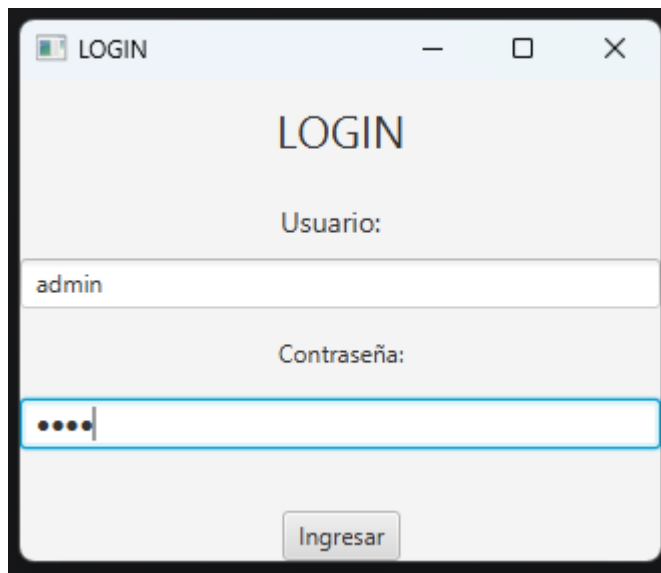
    @FXML
    void mostrarAlerta(String titulo, String mensaje, Alert.AlertType
tipo) {
        Alert alerta = new Alert(tipo);
        alerta.setTitle(titulo);
        alerta.setHeaderText(null);
        alerta.setContentText(mensaje);
        alerta.showAndWait();
    }
}

```

7. Pruebas de Ejecución

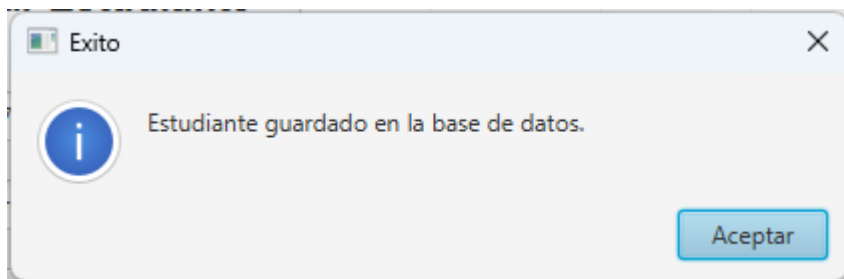
Demostración del sistema funcionando e interactuando correctamente.

1. **Pantalla de Login:**



A screenshot of a web application window titled "LOGIN". The window has a light gray background and a white border. At the top, the title "LOGIN" is centered in a large, bold, black font. Below the title, there are two input fields. The first field is labeled "Usuario:" and contains the text "admin". The second field is labeled "Contraseña:" and contains four black dots, indicating a password. Below the password field, there is a button labeled "Ingresar". The window has standard Windows-style window controls (minimize, maximize, close) in the top right corner.

2. **Alerta de Éxito al Guardar/Modificar:**



3. **Tabla con Datos de PostgreSQL:**

[illegible]